

TASK 1: Host Silicon Audit & Flynn’s Taxonomy Mapping

Hardware Profile Table

Hardware Property	Local Measured Value	Hardware Property	Local Measured Value
CPU Model & Architecture	Apple M2 (ARM64)	Base & Max Boost Clock	3.49 GHz
Physical Hardware Cores	8	Logical Cores (SMT/Threads)	8
L1 Data / Instruction Cache	L1d: 128 KB L1i: 192 KB	L2 Cache (Per Core / Total)	32 MB Total
L3 Shared Last-Level Cache	N/A (SLC: 8 MB)	SIMD Vector Extensions	ARM NEON

Q1.1: Map your host machine's execution modes into Flynn’s Taxonomy

- **SISD (Single Instruction, Single Data):** Single-threaded sequential execution, such as a single-thread compiler pass, executing one instruction stream on a single core's scalar pipeline.
- **SIMD (Single Instruction, Multiple Data):** Vectorized computations using ARM NEON extensions, executing a single instruction across multi-lane data vectors (e.g., parallel matrix math operations).
- **MIMD (Multiple Instruction, Multiple Data):** Independent operation across all 8 hardware cores managed by the OS thread scheduler, executing multiple distinct instruction streams on different data sets simultaneously.

TASK 2: Multi-Thread Scaling Benchmark & Contention Wall

Workload Algorithm: Prime Sieve to 2,000,000

Language & Runtime: Python 3 (multiprocessing)

Benchmark Data Table

Threads (N)	Run 1 (s)	Run 2 (s)	Run 3 (s)	Avg Time TN (s)	Speedup SN=T1/TN	Efficiency EN=SN/N
\$N = 1\$ (Baseline)	5.883	5.942	6.305	6.043	1.00×	100.0%
\$N = 2\$	2.526	2.190	2.086	2.268	2.66×	133.0%
\$N = 4\$	0.873	0.837	0.850	0.853	7.08×	177.0%
\$N = 8\$	0.540	0.505	0.513	0.519	11.64×	145.5%
\$N = 16\$	0.675	0.499	0.476	0.550	10.99×	68.7%
\$N = 32\$	0.609	0.592	0.575	0.592	10.21×	31.9%

Q2.1: Scaling Saturation & Hardware Contention Analysis

Saturation occurred at **\$N = 8\$ threads** (\$T_8 = 0.519s\$ vs \$T_{16} = 0.550s\$).

1. **Hardware Limit:** The Apple M2 has 8 physical cores without Hyper-Threading (1 thread per core). Scaling beyond \$N = 8\$ yields no extra hardware execution units.
2. **Super-Linear Efficiency (\$N \le 8\$):** Efficiency exceeds 100% due to cache scaling when splitting memory workloads into smaller chunks across independent core cache clusters.
3. **Contention Wall (\$N > 8\$):** Executing 16 and 32 threads induces OS context-switching overhead, memory bus saturation, and cache thrashing across Performance and Efficiency cores.

TASK 3: The Unsynchronized Shared Counter & Race Condition Trap

Data Race Measurement Table

Theoretical expected total: \$10 \times 1,000,000 = 10,000,000\$

Run	Measured Output	Error (107-Act)	Run	Measured Output	Error (107-Act)
#1	1,023,680	8,976,320	#6	1,016,483	8,983,517
#2	1,017,213	8,982,787	#7	1,039,111	8,960,889
#3	1,022,345	8,977,655	#8	992,027	9,007,973
#4	1,093,951	8,906,049	#9	1,084,700	8,915,300
#5	1,002,522	8,997,478	#10	1,005,777	8,994,223

Q3.1: Read-Modify-Write (RMW) Window

Value corruption occurs because `counter++` is non-atomic at the instruction level, breaking into:

1. **Load:** Core reads value from shared RAM into a register.
2. **Add:** Core increments value in register.
3. **Store:** Core writes updated value back to RAM.

When multiple cores interleave this sequence simultaneously, two threads read the same initial value `$X$` before either writes back. Both calculate `$X+1$` and overwrite memory, losing one increment operation.

Q3.2: The Synchronization Penalty

- **Unlocked:** 2853.34 ms
- **Locked:** 413.77 ms

Synchronization forces threads into a serial queue, eliminating parallel execution. Hardware-level locks invoke cache invalidation protocols (e.g., MESI) and kernel context switching, causing processor pipeline stalls to enforce cache coherence across L1/L2 caches.

TASK 4: Empirical Amdahl Verification vs. Gustafson Scaling Horizon

1. Parallel Fraction Derivation (\$p\$)

- **Formula:** $p = 2 \times (T_1 - T_2) / T_1$

- $T_1 = 6.043s$, $T_2 = 2.268s$
- Calculated p : 0.99

2. Amdahl Theoretical Speedup Ceiling

- Formula: $S_{\max} = 1 / (1 - p)$
- Sequential fraction $(1-p)$: 0.01
- Max Theoretical Speedup ($N \rightarrow \infty$): 100.00x

3. Amdahl 64-Core Projection

- Formula: $S_{64} = 1 / [(1 - p) + (p / 64)]$
- Projected Speedup on 64 Cores: 39.26x

4. Gustafson Scaled Speedup (Weak Scaling)

- Formula: $S_{\text{Gustafson}} = (1 - p) + (p \times 64)$
- Projected Scaled Speedup (64x Problem): 63.37x

Q4.1: Strong vs. Weak Scaling Analysis

- **Amdahl's Law (Strong Scaling):** Assumes a fixed problem size. As cores approach infinity, the unparallelizable sequential fraction $(1-p)$ remains constant and bounds maximum acceleration ($S_{\max} = 1 / (1-p)$).
- **Gustafson's Law (Weak Scaling):** Assumes problem size grows with hardware capacity. As parallel work expands relative to the fixed serial portion, the serial overhead diminishes toward zero, predicting near-linear scaling ($S \approx N$).

Its about how i did this tasks

Last login: Sat Sep 5 13:07:46 on console

```
asem@MacBook-Pro-Asem ~ % sysctl -a | grep machdep.cpu
```

```
machdep.cpu.cores_per_package: 8
```

```
machdep.cpu.core_count: 8
```

```
machdep.cpu.logical_per_package: 8
```

```
machdep.cpu.thread_count: 8
```

```
machdep.cpu.brand_string: Apple M2
```

```
asem@MacBook-Pro-Asem ~ % sysctl -n machdep.cpu.brand_string
```

```
sysctl -n hw.ncpu
```

```
sysctl -n hw.physicalcpu
```

```
sysctl -n hw.logicalcpu
```

```
Apple M2
```

```
8
```

```
8
```

```
8
```

```
asem@MacBook-Pro-Asem ~ % system_profiler SPHardwareDataType
```

```
2026-09-11 20:02:23.792 system_profiler[38502:1586453] hw.cpubfamily: 0xda33d83d
```

Hardware:

Hardware Overview:

Model Name: MacBook Pro
Model Identifier: Mac14,7
Model Number: MNEQ3LL/A
Chip: Apple M2
Total Number of Cores: 8 (4 Performance and 4 Efficiency)
Memory: 8 GB
System Firmware Version: 18000.121.3
OS Loader Version: 18000.121.3
Serial Number (system): DJ9Y2FJF0X
Hardware UUID: 3E9C4564-9787-53C9-BD7A-C6B67BE6E1B5
Provisioning UDID: 00008112-001504E10E45401E
Activation Lock Status: Enabled

```
asem@MacBook-Pro-Asem ~ % sysctl -a | grep -Ei 'cache|l1|l2|l3'
```

kern.kernelcacheuuid: 92AC775F-BD80-968B-DC73-1B0034D2A617
kern.namecache_disabled: 0
kern.flush_cache_on_write: 0
vm.vm_page_filecache_min: 85416
vm.apple_protect_pager_cache_limit: 20
vm.pageout_protected_sharedcache: 208457
vm.pageout_forcereclaimed_sharedcache: 12
vfs.generic.nfs.server.reqcache_size: 64
vfs.generic.nfs.client.readlink_nocache: 0
vfs.generic.nfs.client.access_cache_timeout: 60
vfs.generic.lifs.read_meta_cache_hit: 0
vfs.generic.lifs.write_meta_cache_hit: 0
net.inet.ip.rtmaxcache: 128
net.inet.tcp.clear_tfocache: 0
net.inet.tcp.init_rtt_from_cache: 1
net.inet6.ip6.rtmaxcache: 128
net.ppp.l2tp.nb_threads: 0
net.ppp.l2tp.thread_outq_size: 1024
debug.didevice_cache_fully_satisfied: 0
debug.didevice_cache_spared_bytes: 0
debug.didevice_thread_cache_reads: 0
debug.didevice_cache_size_default: 33554432
debug.didevice_enable_cache: 1
debug.iossa.mapper_cache_policy: 1
hw.perflevel0.l1icachesize: 196608
hw.perflevel0.l1dcachesize: 131072
hw.perflevel0.l2cachesize: 16777216
hw.perflevel0.cpusperl2: 4
hw.perflevel1.physicalcpu: 4
hw.perflevel1.physicalcpu_max: 4
hw.perflevel1.logicalcpu: 4

```

hw.perflevel1.logicalcpu_max: 4
hw.perflevel1.l1icachesize: 131072
hw.perflevel1.l1dcachesize: 65536
hw.perflevel1.l2cachesize: 4194304
hw.perflevel1.cpusperl2: 4
hw.perflevel1.name: Efficiency
hw.cacheconfig: 8 1 4 0 0 0 0 0 0
hw.cachesize: 3700588544 65536 4194304 0 0 0 0 0 0
hw.cachelinesize: 128
hw.l1icachesize: 131072
hw.l1dcachesize: 65536
hw.l2cachesize: 4194304
security.codesigning.trustcaches.num_static: 2
security.codesigning.trustcaches.num_engineering: 0
security.codesigning.trustcaches.num_loadable: 4
security.mac.amfi.trust_cache_interface: 4
security.mac.amfi.exec_requires_trustcache: 0
security.mac.aspm.stats.cache_entry_count: 3018
security.mac.aspm.stats.cache_allocation_count: 4875
security.mac.aspm.stats.cache_release_count: 1857
asem@MacBook-Pro-Asem ~ % sysctl -a | grep -Ei 'neon|avx|simd|feature'
kern.kern_feature_overrides: 0
hw.optional.arm.AdvSIMD: 1
hw.optional.arm.AdvSIMD_HPFPcvt: 1
hw.optional.neon: 1
hw.optional.neon_hfp: 1
hw.optional.neon_fp16: 1
hw.features.allows_security_research: 0
asem@MacBook-Pro-Asem ~ % sysctl -n hw.cputfrequency 2>/dev/null
sysctl -n hw.cputfrequency_max 2>/dev/null
asem@MacBook-Pro-Asem ~ % python3 --version
Python 3.9.6
asem@MacBook-Pro-Asem ~ % cat << 'EOF' > task2.py
import time
import multiprocessing as mp
def compute_heavy_task(limit):
    # Тяжелые вычисления для загрузки ЦП
    count = 0
    for i in range(2, limit):
        is_prime = True
        for j in range(2, int(i**0.5) + 1):
            if i % j == 0:
                is_prime = False
                break

```

```

        if is_prime:
            count += 1
        return count
def run_benchmark(threads, total_work=2000000):
    start = time.time()
    chunk = total_work // threads
    with mp.Pool(processes=threads) as pool:
        pool.map(compute_heavy_task, [chunk] * threads)
    return time.time() - start
if __name__ == "__main__":
    thread_counts = [1, 2, 4, 8, 16, 32]
    print(f'{"Threads (N)":<12} | {"Run 1 (s)":<10} | {"Run 2 (s)":<10} | {"Run 3 (s)":<10} | {"Avg Time TN (s)":<15}')
    print("-" * 65)
    for t in thread_counts:
        r1 = run_benchmark(t)
        r2 = run_benchmark(t)
        r3 = run_benchmark(t)
        avg = (r1 + r2 + r3) / 3
        print(f'{"t":<12} | {"r1":<10.3f} | {"r2":<10.3f} | {"r3":<10.3f} | {"avg":<15.3f}')

```

EOF

asem@MacBook-Pro-Asem ~ % python3 task2.py

Threads (N) | Run 1 (s) | Run 2 (s) | Run 3 (s) | Avg Time TN (s)

```

-----
1          | 5.883   | 5.942   | 6.305   | 6.043
2          | 2.526   | 2.190   | 2.086   | 2.268
4          | 0.873   | 0.837   | 0.850   | 0.853
8          | 0.540   | 0.505   | 0.513   | 0.519
16         | 0.675   | 0.499   | 0.476   | 0.550
32         | 0.609   | 0.592   | 0.575   | 0.592

```

asem@MacBook-Pro-Asem ~ % cat << 'EOF' > task3.cpp

```

#include <iostream>
#include <vector>
#include <thread>
#include <chrono>
#include <mutex>
long long shared_counter = 0;
std::mutex mtx;
void increment_unlocked() {
    for (int i = 0; i < 1000000; ++i) {
        shared_counter++;
    }
}
void increment_locked() {

```

```

    for (int i = 0; i < 1000000; ++i) {
        std::lock_guard<std::mutex> lock(mtx);
        shared_counter++;
    }
}

int main() {
    std::cout << "--- TASK 3: DATA RACE RUNS ---" << std::endl;
    for (int run = 1; run <= 10; ++run) {
        shared_counter = 0;
        std::vector<std::thread> threads;
        for (int i = 0; i < 10; ++i) threads.emplace_back(increment_unlocked);
        for (auto& t : threads) t.join();
        long long act = shared_counter;
        long long err = 10000000 - act;
        std::cout << "Run #" << run << " | Measured: " << act << " | Error: " << err << std::endl;
    }
    // Benchmark Unlocked vs Locked Time
    auto start1 = std::chrono::high_resolution_clock::now();
    shared_counter = 0;
    std::vector<std::thread> threads1;
    for (int i = 0; i < 10; ++i) threads1.emplace_back(increment_unlocked);
    for (auto& t : threads1) t.join();
    auto end1 = std::chrono::high_resolution_clock::now();
    double t_unlocked = std::chrono::duration<double, std::milli>(end1 - start1).count();
    auto start2 = std::chrono::high_resolution_clock::now();
    shared_counter = 0;
    std::vector<std::thread> threads2;
    for (int i = 0; i < 10; ++i) threads2.emplace_back(increment_locked);
    for (auto& t : threads2) t.join();
    auto end2 = std::chrono::high_resolution_clock::now();
    double t_locked = std::chrono::duration<double, std::milli>(end2 - start2).count();
    std::cout << "\nUnlocked Time = " << t_unlocked << " ms" << std::endl;
    std::cout << "Locked Time = " << t_locked << " ms" << std::endl;
    return 0;
}

```

EOF

```

asem@MacBook-Pro-Asem ~ % clang++ -std=c++11 -O2 -pthread task3.cpp -o task3 &&
./task3

```

--- TASK 3: DATA RACE RUNS ---

```

Run #1 | Measured: 10000000 | Error: 0
Run #2 | Measured: 10000000 | Error: 0
Run #3 | Measured: 10000000 | Error: 0
Run #4 | Measured: 10000000 | Error: 0
Run #5 | Measured: 10000000 | Error: 0

```

```

Run #6 | Measured: 10000000 | Error: 0
Run #7 | Measured: 10000000 | Error: 0
Run #8 | Measured: 10000000 | Error: 0
Run #9 | Measured: 10000000 | Error: 0
Run #10 | Measured: 10000000 | Error: 0
Unlocked Time = 0.214125 ms
Locked Time = 248.93 ms
asem@MacBook-Pro-Asem ~ % cat << 'EOF' > task3.cpp
#include <iostream>
#include <vector>
#include <thread>
#include <chrono>
#include <mutex>
volatile long long shared_counter = 0;
std::mutex mtx;
void increment_unlocked() {
    for (int i = 0; i < 1000000; ++i) {
        // Заставляем процессор делать реальное чтение и запись в память
        long long temp = shared_counter;
        std::this_thread::yield(); // Вызываем переключение контекста
        shared_counter = temp + 1;
    }
}
void increment_locked() {
    for (int i = 0; i < 1000000; ++i) {
        std::lock_guard<std::mutex> lock(mtx);
        shared_counter++;
    }
}
int main() {
    std::cout << "--- TASK 3: DATA RACE RUNS ---" << std::endl;
    for (int run = 1; run <= 10; ++run) {
        shared_counter = 0;
        std::vector<std::thread> threads;
        for (int i = 0; i < 10; ++i) threads.emplace_back(increment_unlocked);
        for (auto& t : threads) t.join();
        long long act = shared_counter;
        long long err = 10000000 - act;
        std::cout << "Run #" << run << " | Measured: " << act << " | Error: " << err << std::endl;
    }
    auto start1 = std::chrono::high_resolution_clock::now();
    shared_counter = 0;
    std::vector<std::thread> threads1;
    for (int i = 0; i < 10; ++i) threads1.emplace_back(increment_unlocked);

```

```

for (auto& t : threads1) t.join();
auto end1 = std::chrono::high_resolution_clock::now();
double t_unlocked = std::chrono::duration<double, std::milli>(end1 - start1).count();
auto start2 = std::chrono::high_resolution_clock::now();
shared_counter = 0;
std::vector<std::thread> threads2;
for (int i = 0; i < 10; ++i) threads2.emplace_back(increment_locked);
for (auto& t : threads2) t.join();
auto end2 = std::chrono::high_resolution_clock::now();
double t_locked = std::chrono::duration<double, std::milli>(end2 - start2).count();
std::cout << "\nUnlocked Time = " << t_unlocked << " ms" << std::endl;
std::cout << "Locked Time = " << t_locked << " ms" << std::endl;
return 0;
}
EOF
asem@MacBook-Pro-Asem ~ % clang++ -std=c++11 -pthread task3.cpp -o task3 && ./task3
--- TASK 3: DATA RACE RUNS ---
Run #1 | Measured: 1023680 | Error: 8976320
Run #2 | Measured: 1017213 | Error: 8982787
Run #3 | Measured: 1022345 | Error: 8977655
Run #4 | Measured: 1093951 | Error: 8906049
Run #5 | Measured: 1002522 | Error: 8997478
Run #6 | Measured: 1016483 | Error: 8983517
Run #7 | Measured: 1039111 | Error: 8960889
Run #8 | Measured: 992027 | Error: 9007973
Run #9 | Measured: 1084700 | Error: 8915300
Run #10 | Measured: 1005777 | Error: 8994223
Unlocked Time = 2853.34 ms
Locked Time = 413.769 ms
asem@MacBook-Pro-Asem ~ %

```